

AHEAD SOLAR

BANGLADESH

Interview Preparation Q&A

Next.js 16 + React 19 • Full-Stack Corporate Website
Comprehensive Technical Reference Guide

85

Questions

16

Sections

2026

Edition

Table of Contents

85 Questions Across 16 Sections

01	Project Overview & Domain	Q1-Q5
02	Tech Stack & Architecture	Q6-Q10
03	Next.js & React Concepts	Q11-Q18
04	Styling & Design System	Q19-Q23
05	Authentication & Security	Q24-Q30
06	API Routes & Data Layer	Q31-Q36
07	File-Based JSON Database	Q37-Q40
08	CMS / Admin Dashboard	Q41-Q46
09	State Management & Data Fetching	Q47-Q51
10	Forms & Validation	Q52-Q55
11	SEO & Performance	Q56-Q62
12	Image & Video Handling	Q63-Q66
13	Animations & UX	Q67-Q71
14	Middleware & Proxy	Q72-Q75
15	Deployment & DevOps	Q76-Q79
16	General / Behavioral Questions	Q80-Q85

1. Project Overview & Domain

Questions 1 - 5

Q1. What is the Ahead Solar BD project about?

Ahead Solar BD is a corporate website for **Ahead Solar**, a solar energy company based in Bangladesh. The platform serves as both a public-facing marketing site and an internal content management system. The public site showcases the company's solar services, completed projects, team members, customer reviews, and provides a contact form for lead generation. A dedicated Palash charging station page highlights a specific EV charging product line.

The admin CMS dashboard allows authorized users to manage all content dynamically - including services, projects, hero slides, team members, reviews, contact inquiries, and site-wide settings - without modifying source code. The system uses a file-based JSON database, keeping deployment simple and free from external database dependencies.

Q2. What are the main public-facing pages of the site?

The public website includes the following key pages organized under the `(main)` route group:

- **Home (/)** - Hero slider, featured services, projects showcase, team section, customer reviews, and a marquee ticker.
- **About (/about)** - Company history, mission, and team overview.
- **Services (/services)** - List of solar energy services with individual detail pages at `/services/[slug]`.
- **Projects (/projects)** - Portfolio of completed installations with detail pages at `/projects/[slug]`.
- **Contact (/contact)** - Contact form for inquiries and a map/address section.
- **Palash Charging Station (/palash-charging-station)** - Dedicated page for the Palash EV charging product with its own color theme.
- **Solutions (/solutions)** - Solutions section accessed via the navbar dropdown.

Additionally, a 404 not-found page and a login page exist outside the main route group.

Q3. What is the Palash charging station page, and how does it differ visually?

The Palash charging station page is a dedicated product page for Ahead Solar's Palash EV charging station. What makes it unique is its **scoped CSS variable override system** defined in `globals.css`. The page applies a `.palash-page` class that completely redefines the brand color palette: the primary cyan is replaced with `#ed1b24` (Palash red), and the forest/deep cyan tones are replaced with `#0d5a4a` (Palash dark green).

This technique leverages Tailwind CSS v4's design token system by overriding the `--color-accent-*`, `--color-gold-*`, and `--color-forest-*` CSS custom properties at the element level. All Tailwind utilities like `text-accent`, `bg-forest-700`, and the `btn-brand` class automatically render in the Palash color scheme without needing entirely new utility classes. The eyebrow label also gets a red palash-flower SVG dot instead of the gold sun icon.

Q4. What type of company is Ahead Solar, and what domain does this project operate in?

Ahead Solar operates in the **renewable energy / solar power** domain in Bangladesh. The company provides solar panel installation, solar energy consulting, and related services. The website positions them as a professional solar solutions provider with a corporate identity featuring a cyan (`#28cbc6`) and gold (`#F7B308`) brand palette that evokes solar energy themes.

The domain influences design decisions: the site uses solar/energy-themed iconography (sun SVGs in section eyebrows), nature-inspired color tones (forest greens, cyan), and a clean professional layout appropriate for B2B and B2C solar energy clients in the Bangladesh market. The inclusion of an EV charging station product (Palash) indicates expansion into electric vehicle infrastructure.

Q5. Who are the target users of this website?

The website serves two distinct audiences:

- **Public Visitors (B2B/B2C):** Homeowners, business owners, and organizations in Bangladesh looking for solar energy solutions. They browse services, view completed projects for social proof, read customer reviews, and submit contact forms for quotes or consultations.
- **Admin Users (Internal Staff):** Ahead Solar employees with `superadmin` or `admin` roles who log into the CMS dashboard to manage all site content. The admin interface uses a dark theme for extended work sessions.

The Palash charging station page also targets EV owners and fleet operators interested in charging infrastructure.

2. Tech Stack & Architecture

Questions 6 - 10

Q6. What is the complete tech stack used in this project?

The tech stack comprises:

- **Framework:** Next.js 16.2.9 with React 19.2.4 and React DOM 19.2.4
- **Language:** TypeScript 5.x
- **Styling:** Tailwind CSS v4 (via `@tailwindcss/postcss`) with custom design tokens defined in CSS
- **Data Fetching:** TanStack React Query v5.101.2
- **Forms:** React Hook Form v7.80 with `@hookform/resolvers` v5.4 and Yup v1.7 validation
- **Rich Text:** Tiptap v3.27 editor (with image and link extensions)
- **Carousel:** Swiper v14.0
- **Icons:** Lucide React v1.21
- **Auth:** JWT (jsonwebtoken v9.0.3) + bcryptjs v3.0.3 for password hashing
- **Notifications:** React Toastify v11.1
- **UI Utilities:** class-variance-authority, clsx, tailwind-merge, Radix UI Slot
- **Database:** File-based JSON (no external DB)
- **Package Manager:** pnpm

Q7. Why was Next.js 16 chosen over other frameworks?

Next.js 16 was chosen for its **full-stack capabilities** - it provides both the frontend rendering and backend API routes in a single project, eliminating the need for a separate server. The App Router offers file-based routing with support for layouts, loading states, and parallel routes, which maps cleanly to the site's page structure.

Next.js 16 also provides built-in features critical for a corporate site: automatic code splitting, server-side rendering (SSR), static site generation (SSG), image optimization, SEO metadata APIs (`generateMetadata`), sitemap and robots.txt generation, and middleware support for authentication. The framework's React 19 compatibility ensures access to the latest concurrent features, and its Turbopack-powered dev server provides fast HMR during development.

Q8. Describe the overall project architecture and folder structure.

The project follows the Next.js App Router convention:

- `src/app/` - Route definitions, layouts, API routes. Contains `(main)/` for public pages, `admin/` for CMS, and `api/` for backend endpoints.
- `src/components/` - Organized by feature: `Home/` , `About/` , `Services/` , `Projects/` , `Contact/` , `Palash/` , `Admin/` , `Auth/` , `Common/` , `SEO/` , `ui/` .
- `src/data/` - TypeScript type definitions and default data. The `api/` subdirectory holds JSON data files used as the database.
- `src/lib/` - Core utilities: auth, token management, file store, API client, queries, caching, rate limiting, image/video helpers, config, and site metadata.
- `src/hooks/` - Custom React hooks like `useInView` .
- `src/types/` - Shared TypeScript interfaces.
- `public/` - Static assets (images, videos, logos).

Q9. What is the role of `src/lib/config.ts`?

`src/lib/config.ts` serves as the **centralized configuration hub** for the application. It exports environment-derived and hardcoded constants including `SITE_URL` (from the `SITE_URL` environment variable, defaulting to `https://aheadsolarbd.com`), social media URLs, company branding constants (`COMPANY_NAME` , `SITE_LOGO` , `ADMIN_LOGO` , `FAVICON`), and the default Open Graph image path.

This file ensures a single source of truth for site-wide constants. It is imported by the layout, metadata generators, API routes, and components, preventing hardcoded URLs and inconsistent branding across the application. The `SITE_URL` is particularly important for generating canonical URLs, sitemaps, and Open Graph metadata.

Q10. How does the project handle environment variables?

The project uses a dedicated `src/lib/env.ts` module that runs at import time and validates required environment variables: `JWT_SECRET` , `JWT_REFRESH_SECRET` , `DEFAULT_SUPERADMIN_EMAIL` , and `DEFAULT_SUPERADMIN_PASSWORD` . If any are missing, it logs a warning in development but **throws an error in production**, preventing the app from starting in a misconfigured state.

The `.env` file at the project root contains the actual secret values. Environment variables are accessed via `process.env` throughout the codebase - in API routes for authentication, in `config.ts` for site URL, and in `token.ts` for JWT signing secrets. The centralized validation approach in `env.ts` prevents scattering `process.env` checks across multiple files.

3. Next.js & React Concepts

Questions 11 - 18

Q11. What is the App Router, and how does this project use it?

The App Router is Next.js's file-system based router introduced in Next.js 13, replacing the Pages Router. This project uses it extensively with route groups: `(main)/` for public pages, `admin/` for the CMS dashboard, and `api/` for backend endpoints. Each folder maps to a URL segment, and special files (`page.tsx`, `layout.tsx`, `loading.tsx`) define the UI.

The root `src/app/layout.tsx` wraps the entire application with `ClientProviders` (for React Query) and `LazyLayout`. The `(main)/layout.tsx` provides the public site shell (navbar, footer), while `admin/layout.tsx` provides the dark-themed admin shell with sidebar navigation. A catch-all route `[...catchAll]` handles dynamic admin sub-routes.

Q12. What is the difference between Server Components and Client Components in this project?

In Next.js 16 with React 19, components are **Server Components by default**. Server Components run on the server, can directly access the file system for reading JSON data, and send minimal HTML to the client. The root `page.tsx` and `layout.tsx` are server components - they call `getSiteInfo()` directly and generate metadata.

Client Components are marked with `"use client"` and run in the browser, handling interactivity, forms, state, effects, and event handlers. Examples include `src/lib/queries.ts`, `src/lib/apiClient.ts`, and `src/hooks/useInView.ts`. The home page uses `dynamic()` to lazy-load the `HomePage` client component, creating a clear boundary between server-rendered shell and interactive client UI.

Q13. How does the `dynamic()` import work in the home page?

The home page uses `next/dynamic` to lazily load the `HomePage` component with a `MainSitePageLoading` skeleton as the loading state. This performs **code splitting** - the `HomePage` and all its children are bundled into a separate JavaScript chunk that loads asynchronously.

The server-rendered page shell (JSON-LD structured data, metadata) is sent immediately, while the interactive home page hydrates in the browser. This is especially beneficial for a large home page with hero sliders, service cards, project galleries, and team sections - only the critical above-the-fold content needs to load first, significantly improving the initial page load performance.

Q14. What is the role of the root layout .tsx?

The root layout performs three key functions:

- **Font Loading:** Loads three Google Fonts via `next/font/google` - Rajdhani (headings), Rubik (body), Geist_Mono (code) - exposing them as CSS custom properties for the Tailwind theme.
- **Global Metadata:** The `generateMetadata()` function reads site info from the JSON database and sets default title template, description, Open Graph, and Twitter card metadata.
- **Provider Wrapping:** Wraps children with `ClientProviders` (React Query, Toastify) and `LazyLayout` for deferred loading.

The HTML element also receives font CSS variables and an `antialiased` class for smooth font rendering.

Q15. What are React 19 features leveraged in this project?

The project uses React 19.2.4, leveraging several improvements:

- **Server Components:** Native support for components that render only on the server, used throughout the app directory.
- **Improved Hydration:** Better handling of server-client mismatches, reducing console errors.
- **ref as a prop:** React 19 allows passing ref as a regular prop instead of using `forwardRef`, simplifying component APIs.
- **Resource Loading:** Built-in support for loading resources (fonts via `next/font`) with automatic preloading.

React 19 also improves concurrent rendering performance, benefiting the complex home page with its multiple data-driven sections and improving overall user experience.

Q16. How do the robots.ts and sitemap.ts work?

`robots.ts` exports a function returning a `MetadataRoute.Robots` object. It allows all crawlers to index the public site but **disallows /admin and /login** paths. It also points to the sitemap URL.

`sitemap.ts` exports a force-dynamic function that builds a sitemap by combining static routes (home, about, services, projects, contact, palash) with dynamic entries read from the JSON data files. Each service and project slug becomes a unique sitemap URL. This ensures new content added via the CMS automatically appears in the sitemap without a rebuild, keeping SEO data synchronized with content changes.

Q17. What is the purpose of ClientProviders and LazyLayout?

`ClientProviders` is a client component that wraps the application with necessary context providers - primarily the `QueryClientProvider` for TanStack React Query and the `ToastContainer` for React Toastify notifications. Since providers use React context (a client-only feature), they must be in a client component, even though the root layout itself is a server component.

`LazyLayout` provides deferred/lazy rendering for children, likely using React 19's features or a custom implementation to defer rendering of below-the-fold content. Together, these two components bridge the server-client boundary - keeping the root layout as a server component while enabling client-side interactivity for the entire application tree.

Q18. What is the purpose of the not-found.tsx file?

The `src/app/not-found.tsx` file defines a custom 404 page that Next.js renders when a user navigates to a URL that doesn't match any defined route. This provides a branded, user-friendly error page instead of the default browser or Next.js 404 page.

Additionally, there is a separate `src/app/admin/not-found.tsx` for the admin section using the dark admin theme. The admin also has a `[...catchAll]` route that acts as a catch-all for unmatched admin paths, providing more granular 404 handling within the CMS dashboard. This dual approach ensures both public visitors and admin users see context-appropriate error pages.

4. Styling & Design System

Questions 19 - 23

Q19. How does Tailwind CSS v4 work in this project?

This project uses **Tailwind CSS v4** via `@tailwindcss/postcss`, which is a major departure from v3. The critical difference: **there is no tailwind.config.js**. Instead, all design tokens are defined directly in CSS using the `@theme` directive inside `globals.css`.

The `@theme` block defines custom color scales (brand, accent, gold, forest, neutral, admin, semantic), typography fonts, border radii, shadows, and transitions. Tailwind v4 reads these CSS custom properties and generates corresponding utility classes like `bg-brand-500`, `text-accent-600`, `rounded-xl`, etc. The `@import "tailwindcss"` directive at the top of `globals.css` activates the Tailwind v4 engine.

Q20. Explain the color token system used in the design.

The design system uses a multi-layered color architecture:

- **Raw Brand Palette:** Primitive color scales defined in `@theme` - `--color-brand-*` (cyan, #28CBC6 primary), `--color-accent-*` (highlights), `--color-gold-*` (secondary accent, #F7B308), `--color-forest-*` (deep cyan for dark sections), `--color-neutral-*` (slate grays), and `--color-admin-*` (dark surface palette).
- **Shorthand Aliases:** `--color-primary` maps to brand-500, enabling utilities like `bg-primary`.
- **Semantic CSS Properties:** In `:root`, `--clr-primary`, `--clr-accent`, `--admin-accent` provide component-level tokens consumed by custom classes.
- **Palash Overrides:** The `.palash-page` scope redefines accent, gold, and forest tokens to red/green values.

Q21. What is the color-mix() function used for in the CSS?

The `color-mix()` CSS function creates semi-transparent variants of brand colors without hardcoding opacity values. For example: `--color-primary-muted: color-mix(in srgb, var(--color-brand-500) 12%, transparent)` creates a 12% opacity version of the brand cyan for background tints.

Similarly, `--shadow-brand` uses `color-mix(in srgb, var(--color-brand-500) 35%, transparent)` for a brand-colored box shadow. This ensures that if the brand color changes (e.g., via the Palash override), all derived colors, shadows, and accents automatically update. It reduces the number of manually defined color variants and maintains consistency across the design system.

Q22. What utility libraries are used alongside Tailwind?

Three utility libraries complement Tailwind CSS:

- **clsx** - A tiny utility for conditionally joining class names. Used when class composition requires logical expressions.
- **tailwind-merge** - Intelligently merges Tailwind class strings without conflicts: `twMerge("p-2 p-4")` results in `"p-4"`. Critical for component libraries where props override base classes.
- **class-variance-authority (CVA)** - Defines component variants with type-safe class mappings. Used for creating consistent, variant-based component APIs.

Together, these enable clean, maintainable class composition without the verbosity and conflict issues of raw Tailwind strings.

Q23. What is the admin dark theme, and how is it implemented?

The admin dashboard uses a dark theme defined through CSS custom properties in `:root`. Key tokens include `--admin-bg: #0f1117`, `--admin-surface: #161b27`, `--admin-surface-2: #1e2535`, and `--admin-surface-hover: #242b3d` for the layered surface hierarchy. Text colors use `--admin-text-primary: #f0f4ff`, `--admin-text-secondary: #8b95b0`, and `--admin-text-muted: #5a6380`.

The accent color is set to the brand teal with glow and muted variants. The admin also maps these tokens to **shadcn-ui compatible variables** (`--card`, `--primary`, `--border`, `--ring`, etc.), enabling compatibility with shadcn-ui component primitives. The admin layout wraps content in a `.admin-layout-root` class that locks body overflow.

5. Authentication & Security

Questions 24 - 30

Q24. How does the JWT authentication system work?

The authentication uses a **dual-token JWT system** implemented in `src/lib/token.ts`:

- **Access Token:** Short-lived (15 minutes), signed with `JWT_SECRET`. Contains user ID, email, and role (superadmin or admin). Used for authenticating API requests.
- **Refresh Token:** Long-lived (7 days), signed with `JWT_REFRESH_SECRET`. Used to obtain new access tokens without re-authentication.

The flow: client sends credentials to `/api/auth/login`, receives both tokens. Access token is stored in `localStorage` and attached as `Authorization: Bearer` header. When it expires (401 response), the `apiClient` automatically calls `/api/auth/refresh` with the refresh token to get a new access token, then retries the original request.

Q25. How does the apiClient handle token refresh?

The `apiClient` at `src/lib/apiClient.ts` implements automatic token refresh with **request deduplication**. When a fetch returns 401, it checks if the failed URL is an auth route (to avoid infinite loops). If not, it calls `refreshOrSingle()`, which ensures only one refresh request is in flight at a time using a shared promise variable.

If refresh succeeds, the original request is retried with the new token. If it fails, `clearAuth()` removes both tokens from `localStorage` and `redirectToLogin()` redirects to `/login` (only if the current path starts with `/admin`). This ensures smooth session management without flickering or data loss during normal browsing.

Q26. How are passwords stored and verified?

Passwords are hashed using **bcryptjs** with a salt rounds value of **12**: `bcrypt.hash(password, 12)`. This provides strong protection against rainbow table and brute-force attacks. The `hashPassword()` and `comparePassword()` functions in `token.ts` wrap `bcrypt`'s async API.

When a user attempts login, the provided password is compared against the stored hash using `bcrypt.compare()`. The default superadmin account is created automatically on first boot via `ensureSuperadminExists()` in `auth.ts`, which reads credentials from environment variables, hashes the password, and saves the user to the JSON data file with a UUID-based ID.

Q27. What roles exist in the system?

The system defines two roles in the `TokenPayload` interface: `"superadmin"` and `"admin"`. Both roles can access the CMS dashboard, but the distinction suggests different permission levels for future implementation.

The `StoredUser` interface includes `id`, `name`, `email`, `password`, `role`, `createdAt`, and `updatedAt` fields. The superadmin role is automatically seeded from environment variables if no superadmin exists. Additional admin users can be created through the CMS user management interface at `/admin/users`. The role is embedded in the JWT payload and can be checked in API routes for role-based access control.

Q28. How does the middleware proxy protect API routes?

The middleware at `src/proxy.ts` intercepts all requests matching `/api/:path*`. It implements a **whitelist-based access control** system:

- **Always public:** Auth routes (login, refresh, logout)
- **Public GET:** Routes like `/api/settings`, `/api/services`, `/api/projects`, `/api/hero-slides`, `/api/team`, `/api/reviews`, `/api/image`, `/api/db`, `/api/env` allow unauthenticated reads
- **Public POST:** Only `/api/contact`, `/api/reviews`, and `/api/palash-applications` allow unauthenticated writes
- **Protected:** All other routes require a valid JWT access token

The middleware extracts the token from either the `Authorization` header or the `accessToken` cookie, verifies it, and returns 401 if invalid.

Q29. How does rate limiting work?

Rate limiting is implemented in `src/lib/rateLimit.ts` using an **in-memory sliding window** approach. The `isRateLimited()` function takes a key (typically the client IP), a max request count (default 10), and a time window (default 15 minutes). It maintains a Map of buckets, each tracking a request count and reset timestamp.

The `getClientIp()` function extracts the real IP from `x-forwarded-for` or `x-real-ip` headers for proxied deployments. Note: this is a single-process rate limiter - multiple server instances would not share state, which is acceptable for this project's scale and deployment model.

Q30. What security measures are in place beyond authentication?

Several security measures extend beyond JWT auth:

- **Path traversal prevention:** Image and video helpers reject URLs containing `..` or backslashes.
- **File validation:** `imageHelper.ts` validates folder names against an allowlist (services, projects, team, hero, settings) and enforces a 5MB size limit.
- **Input sanitization:** Image IDs are validated with regex to prevent injection attacks.
- **Write locks:** `withWriteLock()` in `fileStore.ts` serializes concurrent file writes to prevent data corruption.
- **Environment validation:** `env.ts` fails in production if secrets are missing.
- **Robots.txt:** Disallows indexing of admin and login routes.

6. API Routes & Data Layer

Questions 31 - 36

Q31. What API routes exist in the project?

The `src/app/api/` directory contains the following route groups:

- `auth/` - Login, refresh, and logout endpoints
- `contact/` - Contact form submissions (public POST)
- `db/` - Database operations (public GET)
- `env/` - Environment info (public GET)
- `hero-slides/` - Hero carousel slides (public GET)
- `image/` - Image serving (public GET)
- `palash-applications/` - Palash applications (public POST)
- `projects/` - Project CRUD
- `reviews/` - Reviews (public GET + public POST)
- `services/` - Service CRUD
- `settings/` - Site settings (public GET)
- `team/` - Team member CRUD
- `users/` - User management (admin-only CRUD)

Q32. How do API routes communicate with the frontend?

API routes follow a consistent response pattern: `{ success: boolean, data?: T, error?: string }`.

The frontend uses TanStack React Query hooks defined in `queries.ts` to fetch data. Each hook calls `apiFetchJson()`, which uses the `apiClient` to make authenticated requests.

The `apiClient` automatically attaches the JWT access token from `localStorage` as a Bearer token. If a request returns 401, it attempts a silent token refresh before retrying. The query hooks define `staleTime` of 5 minutes for public data and 0 (always refetch) for admin data. The `queryKeys` object provides consistent cache keys.

Q33. What is the data flow for a public page like the services list?

The data flow for the services page works as follows:

1. Browser requests `/services` .
2. Next.js serves the (main)/layout.tsx shell (navbar, footer) and the services page component.
3. The services page component calls `useQueryServices()` from React Query.
4. React Query's queryFn calls `apiFetchJson("/api/services", [])` .
5. apiClient makes a GET request to `/api/services` (no auth needed - public GET).
6. The API route reads `servicesData.json` via `readDataFile()` and returns the array.
7. React Query caches the result for 5 minutes and renders the services list.

This client-side fetching approach means the page shell loads instantly while data loads asynchronously, with a loading skeleton shown during the fetch.

Q34. What caching strategy is used?

The project uses a **multi-layer caching strategy**:

- **React Query Client Cache:** Public data is cached with `staleTime: 5 minutes` and `refetchOnWindowFocus: false`. Admin data has `staleTime: 0` (always fresh).
- **HTTP Cache Headers:** `cache.ts` defines `PUBLIC_CACHE_HEADERS` with `stale-while-revalidate=600` (10 minutes), allowing browsers to serve stale content while revalidating in the background. Admin routes use `NO_CACHE_HEADERS` with `no-store`.
- **Query Invalidation:** When admin modifies content, `invalidateContentCache()` invalidates both the public and admin query keys, ensuring fresh data on next fetch.

Q35. How does the contact form submission work?

The contact form is a public POST endpoint at `/api/contact` . When a visitor submits the form:

1. Client-side validation runs via React Hook Form + Yup (name, email, phone, message fields).
2. Validated data is sent via the apiClient POST to `/api/contact`.
3. The middleware allows this without authentication (it is in `PUBLIC_POST_PATHS`).
4. The API route may apply rate limiting using `isRateLimited()` with the client IP.
5. The submission is saved to `contactData.json` via `writeDataFile()` .
6. A success response is returned, and the admin can view submissions in the `/admin/contact` dashboard.

Q36. How are API responses structured?

All API routes follow a **consistent JSON response format**:

- **Success:** `{ success: true, data: T }` - where T is the resource type (array or object).
- **Error:** `{ success: false, error: "Error message" }` - with appropriate HTTP status code (400, 401, 404, 500).

The frontend's `apiFetchJson()` helper checks `json.success && Array.isArray(json.data)` and falls back to a default value if the response is invalid. This consistent structure simplifies error handling and makes it easy to add global response interceptors. The API client also handles non-JSON responses gracefully.

7. File-Based JSON Database

Questions 37 - 40

Q37. How does the file-based JSON database work?

The database is implemented in `src/lib/fileStore.ts` using Node.js fs module. JSON files are stored in `src/data/api/` with names like `servicesData.json`, `projectsData.json`, etc. The `readDataFile<T>(fileName, fallback)` function reads and parses a JSON file, returning the typed data or a fallback value if the file doesn't exist or is empty.

The `writeDataFile<T>(fileName, data)` function writes data as formatted JSON. A critical feature is `withWriteLock()` - a promise-based mutex that serializes write operations to prevent data corruption from concurrent requests. This is essential because Node.js is single-threaded but async operations can interleave.

Q38. What data files exist in the `src/data/api/` directory?

The `src/data/api/` directory contains JSON files for each content type:

- `usersData.json` - Admin/superadmin user accounts with hashed passwords
- `teamData.json` - Team member profiles
- Additionally, runtime-created files for: services, projects, hero-slides, reviews, contact submissions, settings, and palash applications

Default data is defined in TypeScript files under `src/data/`. When a JSON data file doesn't exist, the API routes fall back to these defaults. The JSON files are created on first write and serve as the persistent storage layer.

Q39. What are the advantages and disadvantages of this approach?

Advantages:

- **Simplicity:** No external database setup, connection strings, or ORM configuration needed.
- **Portability:** The entire database is just JSON files that can be backed up by copying a directory.
- **Performance:** Reads are fast (in-process file I/O) for small to medium datasets.
- **Cost:** No database hosting fees - ideal for a small business site with moderate traffic.

Disadvantages:

- **Scalability:** Not suitable for high-concurrency write workloads; the write lock serializes all mutations.
- **Atomicity:** No transaction support - crashes during writes could corrupt data.
- **Query capability:** No indexing or query optimization; all reads load the full file.
- **Multi-instance:** Multiple server instances would have separate data copies (not shared).

Q40. How does the write lock mechanism prevent data corruption?

The `withWriteLock()` function in `fileStore.ts` implements a **promise-based mutex**. It maintains a module-level `writeLock` variable that holds the latest write promise. When `withWriteLock(fn)` is called, it chains `fn` onto the existing promise via `writeLock.then(fn)`. This ensures that even if multiple API requests try to write simultaneously, their operations execute sequentially.

For example, if two requests try to update services at the same time: the first starts writing immediately, and the second waits until the first's promise resolves (success or failure). The `.then(() => undefined, () => undefined)` ensures the lock chain continues even if a write fails. This prevents the classic race condition where two concurrent reads followed by two writes would cause one write to overwrite the other's changes.

8. CMS / Admin Dashboard

Questions 41 - 46

Q41. What content can be managed through the admin CMS?

The admin CMS at `/admin` provides management interfaces for all dynamic content:

- **Hero Slides (/admin/hero-slider)** - Manage the homepage carousel slides with images, text, and links.
- **Services (/admin/services)** - Create, edit, delete solar service offerings with images, descriptions, and slugs.
- **Projects (/admin/projects)** - Manage the project portfolio with images, descriptions, and slugs.
- **Team (/admin/team)** - Team member profiles with photos and bios.
- **Reviews (/admin/reviews)** - Customer testimonials and ratings.
- **Contact (/admin/contact)** - View and manage contact form submissions.
- **Settings (/admin/settings)** - Global site settings (company info, SEO, social links).
- **Users (/admin/users)** - Manage admin and superadmin accounts.
- **Analytics (/admin/analytics)** - Dashboard analytics and metrics.

Q42. How is the admin layout structured?

The admin layout at `src/app/admin/layout.tsx` provides a dark-themed shell with:

- **Sidebar Navigation:** Fixed 260px wide sidebar with links to each CMS section.
- **Header:** 64px tall header bar with user info and logout.
- **Main Content Area:** Scrollable content area with the dark surface background.
- **Root Class:** The layout applies `.admin-layout-root` which locks body overflow via the CSS selector `html:has(.admin-layout-root) { overflow: hidden !important; }`.

The admin CSS file at `src/app/admin/admin.css` contains additional admin-specific styles. The layout uses the admin dark theme tokens defined in `:root`.

Q43. How does admin data fetching differ from the public site?

Admin queries use separate React Query hooks prefixed with `useAdminQuery*` and use `adminQueryKeys`. The key differences:

- **Stale Time:** Admin queries have `staleTime: 0` and `refetchOnMount: true`, ensuring the admin always sees the latest data. Public queries cache for 5 minutes.
- **Refetch Behavior:** Admin queries skip `refetchOnWindowFocus` to avoid unnecessary requests during editing.
- **Cache Separation:** Public and admin data use different query keys, so invalidating admin data doesn't force a refetch on the public site until content is published.
- **Write Operations:** Admin API routes require authentication (PUT, DELETE), while public routes are read-only.

Q44. How does the Tiptap rich text editor integrate into the CMS?

The CMS uses **Tiptap v3.27** with the following extensions: starter-kit for basic formatting (bold, italic, headings, lists, blockquote, code blocks), extension-image for inline image insertion, and extension-link for hyperlink creation and editing.

Tiptap is used for service descriptions, project descriptions, and other rich content fields. It provides a WYSIWYG editing experience that outputs clean HTML. The editor integrates with React Hook Form for validation and stores content as HTML strings in the JSON data files. The rich text output is rendered on the public site within the page content areas.

Q45. How does the Palash charging station management work in the admin?

The admin interface includes a `/admin/palash-applications` section for managing Palash charging station applications. The palash-applications API route is listed as a public POST endpoint, meaning visitors can apply for a Palash station without authentication.

The public Palash page at `/palash-charging-station` includes an application form that POSTs to this endpoint. The admin can then view and manage these applications in the CMS. The Palash page uses the special `.palash-page` CSS class that overrides the entire color scheme to match the Palash brand (red and dark green), demonstrating the flexibility of the CSS custom property-based design system.

Q46. How does the CMS handle image uploads?

Image uploads are handled by `src/lib/imageHelper.ts`. The flow works as follows: the admin selects an image in the CMS form, client-side JavaScript converts the image to a base64 data URL, and the base64 string is sent to the API route along with the form data.

The `saveImage()` function validates the folder name against an allowlist (services, projects, team, hero, settings), extracts the file type, validates the extension, checks the 5MB size limit, and creates a unique filename with a timestamp. The image buffer is written to

`public/images/api/{folder}/{id}_{timestamp}.{ext}`. The stored URL is saved in the JSON data file.

`deleteImage()` removes the physical file when content is deleted, with security checks for path traversal.

9. State Management & Data Fetching

Questions 47 - 51

Q47. Why is TanStack React Query used instead of native fetch?

TanStack React Query v5 provides several critical features that raw fetch lacks:

- **Automatic Caching:** Results are cached by query key, preventing duplicate network requests for the same data.
- **Stale-While-Revalidate:** Shows cached data immediately while fetching fresh data in the background.
- **Background Refetching:** Can refetch on window focus, interval, or when the component remounts.
- **Query Invalidation:** When admin updates content, `invalidateQueries()` triggers a refetch across all components using that data.
- **Loading/Error States:** Built-in `isLoading`, `isError`, error states for each query.

This is essential for a CMS where content changes frequently and both public and admin views need to stay synchronized.

Q48. What is the difference between queryKeys and adminQueryKeys?

queryKeys defines cache keys for the public site: ["settings"], ["services"], ["projects"], etc. These queries have 5-minute stale time and are consumed by public-facing components.

adminQueryKeys defines prefixed cache keys: ["admin", "settings"], ["admin", "services"], etc. These queries have 0 stale time (always fresh) and are consumed by CMS components.

This separation is deliberate: when an admin updates a service, `invalidateContentCache("services")` invalidates both ["services"] and ["admin", "services"], ensuring both views refresh. But the public site doesn't refetch on every admin page load - only when data is actually modified.

Q49. How does the useInvalidateAll hook work?

The `useInvalidateAll()` hook in `queries.ts` returns a function that, when called, invalidates **all public query keys**: settings, services, hero-slides, team, projects, contact, reviews, and users. It uses React Query's `queryClient.invalidateQueries()` with each key.

This is likely used after a bulk operation (like importing data or resetting the database) to force a complete refetch of all cached data. For individual content updates, the more targeted `invalidateContentCache(queryClient, key)` function is used instead, which only invalidates the specific content type (both public and admin variants).

Q50. How does the useInView hook work?

The `useInView()` hook at `src/hooks/useInView.ts` uses the **Intersection Observer API** to detect when an element enters the viewport. It accepts options for `once` (default true), `threshold` (0.15 default), and `rootMargin` ("0px 0px -8% 0px" default, triggering slightly before the element is fully visible).

It returns a ref to attach to the target element and an `inView` boolean state. When the element intersects, `inView` becomes true, and if `once` is true, the observer is disconnected. If `IntersectionObserver` is unavailable (SSR or old browsers), it falls back to `requestAnimationFrame` to immediately set `inView` to true. This hook powers the scroll reveal animations throughout the site.

Q51. What are the query hook naming conventions?

The project follows a clear naming convention for React Query hooks:

- **Public hooks:** `useQuery{Resource}()` - e.g., `useQueryServices()`, `useQueryProjects()`, `useQueryTeam()`. These use `queryKeys` and 5-minute stale time.
- **Admin hooks:** `useAdminQuery{Resource}()` - e.g., `useAdminQueryServices()`. These use `adminQueryKeys` and zero stale time.
- **Specialized hooks:** `useQueryHeroSlides(site?)` accepts an optional parameter, extending the query key with the site filter.
- **Invalidation hooks:** `useInvalidateAll()` returns an invalidation function.

This consistent naming makes it easy to identify which hooks are safe for public components and which require auth.

10. Forms & Validation

Questions 52 - 55

Q52. How does React Hook Form integrate with Yup in this project?

The project uses `react-hook-form v7.80` with `@hookform/resolvers v5.4` and `yup v1.7` for schema-based validation. The integration works as follows:

1. A Yup schema is defined with field-level validations (required, email, min/max length, regex patterns).
2. The `yupResolver(schema)` from `@hookform/resolvers/yup` bridges Yup's validation to React Hook Form's validation system.
3. The `useForm()` hook is initialized with the resolver, providing `register()`, `handleSubmit()`, `formState: { errors }`.
4. Error messages from Yup are automatically mapped to `formState.errors` and can be displayed next to form fields.

This approach provides type-safe, declarative validation schemas that are shared between client-side display and potentially server-side validation.

Q53. What forms exist in the application?

Forms exist in both the public site and admin CMS:

- **Contact Form (/contact):** Collects name, email, phone, subject, and message. Publicly accessible, submits via POST to /api/contact.
- **Palash Application Form:** Application form for Palash charging station interest. Publicly accessible POST to /api/palash-applications.
- **Login Form (/login):** Email and password authentication form, submits to /api/auth/login.
- **Admin CMS Forms:** Create/edit forms for services, projects, team members, hero slides, reviews, settings, and users. These use Tiptap for rich text fields and file inputs for image uploads.

Q54. How are form errors displayed to the user?

Form errors are displayed using React Hook Form's `formState.errors` object and React Toastify for submission feedback:

- **Field-level errors:** When validation fails, error messages appear below the corresponding field via `errors.fieldName?.message`.
- **Submission feedback:** React Toastify provides toast notifications for success (`toast.success()`) and error (`toast.error()`) states after form submission.
- **Visual styling:** Admin forms use the admin theme's danger color for error text, with Tailwind utilities.

The combination provides immediate field-level feedback and persistent submission status notifications.

Q55. How does client-side validation differ from server-side validation?

Client-side validation (React Hook Form + Yup) runs in the browser before any network request. It provides instant feedback, improves UX by avoiding unnecessary server calls, and reduces server load. However, it can be bypassed by malicious users.

Server-side validation happens in the API route handlers. API routes validate the incoming data structure, check required fields, validate email formats, and ensure data integrity before writing to the JSON files. The server also applies rate limiting and authentication checks.

The project implements both layers: client-side for UX and server-side for security. For example, the contact form validates fields in the browser, but the API route also validates before saving.

11. SEO & Performance

Questions 56 - 62

Q56. How does the project handle SEO metadata?

SEO metadata is handled through Next.js's Metadata API at multiple levels:

- **Root Layout:** `generateMetadata()` in `src/app/layout.tsx` reads site info from the JSON database and sets the default title template (%s | Ahead Solar), description, Open Graph, and Twitter card metadata.
- **Page-level:** The `pageMetadata()` helper in `site.ts` generates per-page metadata with canonical URLs, custom titles/descriptions, and images.
- **Dynamic Content:** Service and project detail pages generate metadata from their content data.

The metadata includes `metadataBase` for resolving relative URLs, `alternates.canonical` for canonical URLs, and structured Open Graph/Twitter data for social sharing.

Q57. What is the JSON-LD structured data used for?

The home page includes two JSON-LD structured data schemas rendered via the `JsonLd` component:

- **Organization Schema:** Contains the company name, URL, logo, description, email, phone, address (with Bangladesh country code), and social media profiles. This helps Google display a Knowledge Panel for the company.
- **WebSite Schema:** Basic website information that helps search engines understand the site structure.

JSON-LD is the preferred format for structured data because it doesn't interfere with the page's CSS or JavaScript. It is embedded in a script tag with `type="application/ld+json"` and parsed by search engines to generate rich results.

Q58. What SEO-friendly features are built into the site?

The site includes several SEO-friendly features:

- **Dynamic Sitemap:** `sitemap.xml` is generated dynamically from both static routes and CMS content (services and projects slugs).
- **Robots.txt:** Generated via `robots.ts`, allowing public pages while blocking `/admin` and `/login`.
- **Canonical URLs:** Each page has a canonical URL set via `alternates.canonical` in metadata.
- **Open Graph & Twitter Cards:** Social media preview data for every page with custom images.
- **Semantic HTML:** Proper heading hierarchy and semantic elements for screen readers and crawlers.
- **Server-Side Rendering:** Pages are rendered on the server, ensuring crawlers see fully rendered HTML.

Q59. How does the project optimize page load performance?

Performance optimizations include:

- **Code Splitting:** `dynamic()` imports split the `HomePage` into a separate chunk, loading only what's needed initially.
- **Font Optimization:** `next/font/google` preloads fonts and uses `font-display: swap` to prevent invisible text.
- **Image Optimization:** Images are served from `public/images/api/` with Next.js's built-in `Image` component providing lazy loading and responsive sizing.
- **React Query Caching:** 5-minute stale time prevents refetching data that hasn't changed.
- **Lazy Loading:** `LazyLayout` defers rendering of below-the-fold content.
- **Reduced Motion:** The CSS includes `@media (prefers-reduced-motion: reduce)` that disables all animations for accessibility.

Q60. What are the cache control headers used?

The project defines two sets of cache headers in `src/lib/cache.ts`:

- **PUBLIC_CACHE_HEADERS:** `Cache-Control: public, max-age=0, must-revalidate, stale-while-revalidate=600` - This tells the browser to always revalidate with the server but allows serving stale content for up to 10 minutes while revalidation happens in the background. Ideal for content that changes occasionally.
- **NO_CACHE_HEADERS:** `Cache-Control: no-store` - Used for admin routes and sensitive data that should never be cached by the browser or intermediate proxies.

The `stale-while-revalidate` pattern provides an excellent balance between freshness and performance for a CMS-driven site.

Q61. How does lazy loading improve the user experience?

Lazy loading is implemented at multiple levels in this project. The `LazyLayout` component wraps the application tree and likely defers rendering of non-critical UI sections until they're needed. The `dynamic()` import on the `HomePage` splits the main content into a separate chunk that loads asynchronously after the shell renders.

On individual pages, the `useInView` hook triggers scroll reveal animations only when elements enter the viewport, and images use Next.js's `Image` component for lazy loading. This progressive loading approach ensures the initial paint is fast and interactive, with content appearing smoothly as the user scrolls down.

Q62. What is the MainSitePageLoading component?

The `MainSitePageLoading` component at `src/components/Common/MainSitePageLoading.tsx` is a **loading skeleton** displayed while the `HomePage` chunk loads asynchronously. It's used as the loading option in the `dynamic()` import on the home page.

This component likely renders placeholder shapes (gray boxes or shimmer effects) that approximate the layout of the home page sections. This provides visual feedback during the loading phase and prevents layout shift when the actual content renders. It improves perceived performance by showing something meaningful immediately rather than a blank page.

12. Image & Video Handling

Questions 63 - 66

Q63. How are images stored and served?

Images are stored as physical files in the `public/images/api/` directory, organized by content type (services, projects, team, hero, settings). When an admin uploads an image via the CMS, the client converts it to base64, sends it to the API, and `saveImage()` in `imageHelper.ts` decodes it back to a buffer and writes it to disk with a unique filename.

The stored URL (e.g., `/images/api/services/service1_1234567890.jpg`) is saved in the corresponding JSON data file. These URLs are relative to the public directory and served directly by Next.js static file serving.

Q64. What image formats and size limits are supported?

The `imageHelper.ts` defines support for these formats through the `EXTENSION_MAP`: JPEG (.jpg), PNG (.png), GIF (.gif), WebP (.webp), and SVG (.svg).

The maximum file size is **5MB** ($5 * 1024 * 1024$ bytes). Files exceeding this limit are rejected with an error before any disk write occurs. Unsupported file types are rejected during the extension mapping step.

Q65. How does video handling work?

Video handling follows a similar pattern to images, implemented in `src/lib/videoHelper.ts`. Videos are stored in the `public/video/` directory. Supported formats include MP4, WebM, OGG/OGV, MOV, and M4V.

The `saveVideo()` function accepts base64 video data, validates the file type, decodes to buffer, and writes to disk. It supports both auto-generated filenames (with timestamp) and sanitized original filenames. The `deleteVideo()` function removes the file with path traversal checks.

Q66. How does the videoUrl helper work?

The `src/lib/videoUrl.ts` module provides utility functions for working with video URLs, handling URL normalization, validation, and potentially YouTube/Vimeo embed URL conversion. This helper ensures consistent video URL handling across the application.

The `videoHelper.ts` module focuses on file system operations (save/delete), while `videoUrl.ts` focuses on URL manipulation. This separation keeps the codebase organized with file operations in one module and URL logic in another.

13. Animations & UX

Questions 67 - 71

Q67. How do scroll reveal animations work?

Scroll reveal animations are driven by the `Reveal` component and the `useInView` hook. The CSS defines a `.reveal` class with initial opacity: 0 and transform: `translate3d(0, 48px, 0)`. When the element enters the viewport via `IntersectionObserver`, the class `.reveal-revealed` is added, triggering a CSS transition to visible state.

Multiple animation variants are supported via data attributes: `fade`, `fade-down`, `zoom`, `slide-left`, `slide-right`, and `scale`. The `cubic-bezier(0.22, 1, 0.36, 1)` timing function creates a smooth, natural-feeling deceleration effect across all variants.

Q68. What is the Ken Burns effect used for?

The Ken Burns effect is a **slow zoom animation** applied to hero slide background images. Defined as a CSS keyframe animation, it scales the image from 1.05 to 1.14 over 9 seconds with an ease-out timing function. This creates a subtle, cinematic zoom that adds visual interest to the hero carousel.

The animation is applied to the `.hero-bg-zoom` element within active Swiper slides via the `.swiper-slide-active .hero-bg-zoom` selector. Each new slide triggers a fresh Ken Burns animation, creating a dynamic, professional-looking hero section common on corporate websites.

Q69. How do the hero text animations work?

The hero section uses a multi-layered animation system:

- **Hero Anime Items:** Elements with class `.hero-anime-item` start at opacity: 0, translateY(34px). The active slide items transition to visible with staggered delays (0.12s, 0.24s, 0.36s, 0.48s for subsequent children).
- **Word-by-Word Reveal:** The `.anime-word` class animates individual words with opacity and transform transitions, creating a typewriter-like reveal effect.
- **CSS-Only Triggering:** All animations are triggered by the `.swiper-slide-active` parent class, replaying with each new slide using pure CSS without JavaScript state management.

Q70. What is the marquee/ticker animation?

The marquee animation creates an **infinite horizontal scrolling ticker** effect used in the footer or between sections. It uses a CSS keyframe that translates the track by -50% since the content is duplicated for seamless looping. The animation runs for 28 seconds with a linear timing function.

The marquee pauses on hover via `animation-play-state: paused`. The content structure uses duplicated groups so when the first set scrolls out of view, the duplicate seamlessly takes its place, creating infinite scrolling without any JavaScript.

Q71. What ambient animations exist in the design system?

The design system includes several ambient motion effects:

- **Float Animation:** `.animate-float` creates a gentle vertical bobbing motion over 5 seconds.
- **Pulse Ring:** `.pulse-ring` creates an expanding, fading ring effect for attention-grabbing CTAs.
- **Gradient Shift:** `.animate-gradient` animates background-position for gradient backgrounds.
- **Card Shine:** `.card-shine` adds a diagonal light sweep across cards on hover.
- **Nav Link Sweep:** `.nav-link-sweep` animates an underline from right-to-left on hover.
- **Chat Widget Animations:** Entrance, button pop-in, and idle bobbing for the floating chat widget.

All animations respect the `prefers-reduced-motion: reduce` media query for accessibility compliance.

14. Middleware & Proxy

Questions 72 - 75

Q72. What is the role of the proxy middleware?

The middleware at `src/proxy.ts` serves as the **API gateway** for the entire application. It intercepts all requests matching `/api/:path*` and implements authentication and authorization logic before any API route handler executes. Non-API requests pass through directly via `NextResponse.next()`.

The middleware uses a whitelist approach: specific routes are marked as public (GET/POST), while everything else requires a valid JWT access token. This protects admin operations (CRUD on services, projects, team, etc.) while keeping public content accessible to search engines and anonymous visitors.

Q73. How does the middleware extract and validate tokens?

The `getToken()` function in `proxy.ts` checks two sources for the JWT token:

1. **Authorization Header:** Looks for `Authorization: Bearer <token>` in the request headers.
2. **Cookies:** Falls back to reading the `accessToken` cookie from the request.

If a token is found, `verifyAccessToken(token)` is called, which uses `jsonwebtoken` to verify the signature against `JWT_SECRET` and decode the payload. If verification fails (expired, invalid signature, malformed payload), the middleware returns a 401 JSON response. This dual-source approach supports both API clients and browser-based requests.

Q74. Why are some routes public while others require authentication?

The route access policy is designed around the **public content vs. admin operations** distinction:

- **Public GET routes:** Services, projects, team, reviews, hero-slides, and settings must be publicly readable for the website to render content. Search engines need to crawl these routes.
- **Public POST routes:** Contact form, reviews, and palash-applications allow anonymous submissions, which is essential for lead generation and social proof.
- **Protected routes:** All write operations (PUT, DELETE) on content and user management require authentication to prevent unauthorized modifications.

This separation ensures the public site works without login while the CMS remains secure.

Q75. What happens when the middleware rejects a request?

When the middleware rejects a request (no token or invalid token), it returns a **401 Unauthorized** JSON response with the structure: `{ success: false, error: "Authorization header missing or invalid" }` or `{ success: false, error: "Invalid or expired token" }`.

The client-side `apiClient` detects the 401 status and automatically attempts a token refresh using the refresh token. If refresh succeeds, the original request is retried with the new token transparently. If refresh also fails (expired refresh token), the client clears stored tokens and redirects to the login page. This creates a seamless authentication experience where users are only prompted to log in when their session truly expires.

15. Deployment & DevOps

Questions 76 - 79

Q76. How is this project deployed?

The project is a Next.js application that can be deployed to any Node.js hosting platform. Given the file-based JSON database, deployment must use a **persistent filesystem** (not a serverless/edge runtime) to ensure data files survive across requests. Common deployment targets include a VPS (DigitalOcean, AWS EC2), a Node.js PaaS (Railway, Render), or a Docker container.

The `pnpm build` command (via `next build`) creates an optimized production build, and `pnpm start` (via `next start`) runs the production server. Environment variables (JWT secrets, admin credentials) must be configured in the hosting environment.

Q77. What environment variables are required for production?

The following environment variables are required for production deployment (validated by env.ts):

- **JWT_SECRET:** A long, random string used to sign access tokens (15-minute expiry).
- **JWT_REFRESH_SECRET:** A separate long, random string for signing refresh tokens (7-day expiry).
- **DEFAULT_SUPERADMIN_EMAIL:** Email for the initial superadmin account.
- **DEFAULT_SUPERADMIN_PASSWORD:** Password for the initial superadmin account (hashed with bcrypt, 12 rounds).

Optional variables include NODE_ENV (defaults to development) and SITE_URL (defaults to <https://aheadsolarbd.com>). In production, missing required variables cause the application to throw an error and refuse to start.

Q78. What are the build scripts and their purposes?

The package.json defines four scripts:

- **pnpm dev** (next dev) - Starts the development server with Turbopack for fast HMR. Supports hot reloading of components, API routes, and CSS changes.
- **pnpm build** (next build) - Creates an optimized production build. Analyzes the bundle, tree-shakes unused code, generates static pages where possible, and outputs to the .next directory.
- **pnpm start** (next start) - Starts the production server using the optimized build. Must be run after `pnpm build`.
- **pnpm lint** (eslint) - Runs ESLint to check for code quality issues, unused imports, and potential bugs. Uses the eslint-config-next preset.

Q79. What are the limitations of the file-based database for production?

The file-based JSON database has several production limitations:

- **No horizontal scaling:** Multiple server instances would each have their own copy of the data, leading to inconsistencies. A single-server deployment is required.
- **Write concurrency:** The write lock serializes all mutations, creating a bottleneck under high write load. For a corporate site with occasional CMS updates, this is acceptable.
- **No backups automation:** There is no built-in backup mechanism. Data can be lost if the filesystem is corrupted or the server crashes during a write. Regular filesystem backups are essential.
- **No querying:** All data is loaded into memory for every read. With a large dataset, this could become slow. For this project's scale (hundreds of records), performance is fine.

For a small-to-medium corporate website, these limitations are acceptable trade-offs for the simplicity and zero-cost deployment benefits.

16. General / Behavioral Questions

Questions 80 - 85

Q80. Why did you choose a file-based database over MongoDB or PostgreSQL?

The file-based JSON database was chosen to **minimize deployment complexity and cost** for a corporate website. Ahead Solar is a small-to-medium business that needs a website to showcase services and collect leads, not handle millions of transactions. Adding MongoDB or PostgreSQL would require additional hosting costs, database management, backup strategies, and connection configuration.

The JSON approach means the entire application and its data can be deployed on a single \$5-10/month VPS with no external dependencies. The data model is simple (flat arrays of records), the read/write patterns are low-volume, and the write lock provides sufficient concurrency control for a CMS with a handful of admin users.

Q81. What challenges did you face during development?

Key challenges included:

- **Next.js 16 breaking changes:** As noted in AGENTS.md, Next.js 16 has breaking changes from earlier versions. Understanding the new App Router conventions, middleware patterns, and API route structure required careful reading of the documentation.
- **Server-Client boundary:** Correctly splitting code between server components (for data fetching and SEO) and client components (for interactivity) required understanding React 19's rendering model.
- **Concurrent write safety:** Implementing the write lock to prevent data corruption from simultaneous admin operations was a non-trivial concurrency challenge in Node.js's async environment.
- **Token refresh flow:** Building a robust auto-refresh system that handles edge cases (expired refresh tokens, concurrent requests during refresh, network failures) required careful implementation.

Q82. How would you improve this project if you had more time?

Several improvements would enhance the project:

- **Database migration:** Replace the JSON file store with SQLite or PostgreSQL for better concurrency, atomicity, and query capabilities.
- **Image optimization:** Integrate Next.js Image component with a CDN for automatic format conversion (WebP/AVIF), responsive sizing, and edge caching.
- **Testing:** Add unit tests for API routes and utilities, integration tests for the auth flow, and E2E tests with Playwright for critical user journeys.
- **Role-based permissions:** Implement granular permissions so admin users can be restricted to specific CMS sections (e.g., a user who can only manage services).
- **Real-time updates:** Use Server-Sent Events or WebSockets to push content changes to the public site without requiring a page refresh.

Q83. Describe your approach to testing this application.

A comprehensive testing strategy for this project would include:

- **Unit Tests:** Test utility functions in isolation - token generation/verification, file store read/write, rate limiting logic, image/video helpers, and form validation schemas. Use Jest or Vitest.
- **API Route Tests:** Test each API endpoint with mock file system operations - verify authentication checks, response formats, error handling, and data validation.
- **Component Tests:** Test React components with React Testing Library - form submissions, loading states, error displays, and interactive elements.
- **E2E Tests:** Use Playwright to test critical user flows - login, content editing, form submission, and page navigation.
- **Visual Regression:** Capture screenshots of key pages and compare against baselines to catch unintended styling changes.

Q84. How does this project handle accessibility?

The project addresses accessibility in several ways:

- **Reduced Motion:** The CSS includes a comprehensive `@media (prefers-reduced-motion: reduce)` block that disables all animations, transitions, and transforms for users who have requested reduced motion in their OS settings.
- **Semantic HTML:** The use of proper heading hierarchy, landmarks, and semantic elements provides structure for screen readers.
- **Font Legibility:** The `antialiased` class on the HTML element ensures smooth font rendering, and the font sizes are appropriate for readability.
- **Focus Indicators:** Tailwind's default focus ring styles provide visible focus indicators for keyboard navigation.
- **Color Contrast:** The design token system uses carefully chosen color values that maintain sufficient contrast ratios for text readability.

Q85. What did you learn from building this project?

Key learnings from the Ahead Solar BD project:

- **Full-Stack Next.js Mastery:** Deep understanding of the App Router, server/client component boundaries, API routes, middleware, and metadata generation in Next.js 16.
- **Design System Architecture:** Building a multi-theme CSS custom property system that supports brand switching (cyan/gold for main site, red/green for Palash, dark theme for admin) using Tailwind v4's token architecture.
- **Authentication Patterns:** Implementing a production-quality JWT auth system with automatic token refresh, request deduplication, and middleware-based route protection.
- **Simplified Data Layer:** Understanding when a file-based database is appropriate and how to add concurrency safety (write locks) to prevent data corruption.
- **Performance Optimization:** Applying code splitting, lazy loading, scroll animations, and caching strategies to create a fast, engaging user experience.